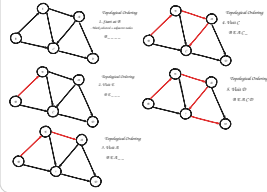


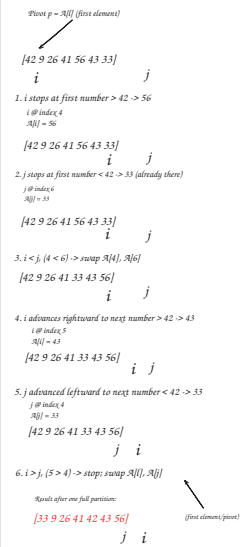
Algorithm Design — Diagram Reference (Hand-drawn)

Print double-sided: diagrams this side, explanations on the back.

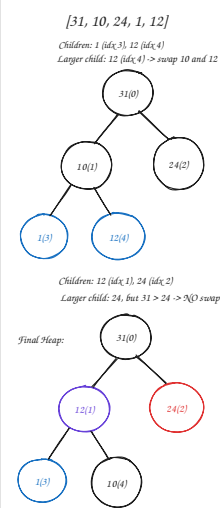
1 · Topological Sort



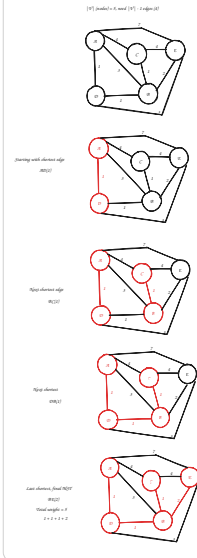
2 · Hoare Partition



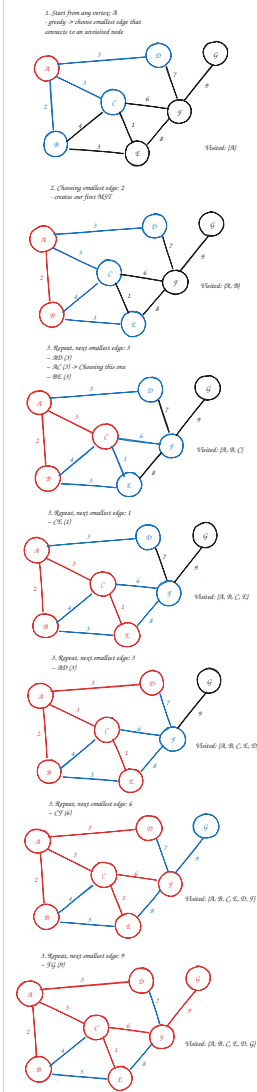
3 · Bottom-up Heapify



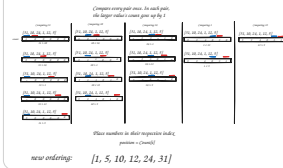
4 · Kruskal MST



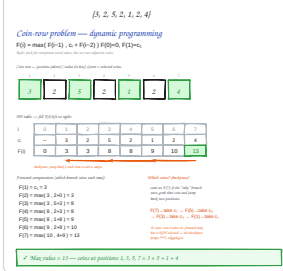
5 · Prim MST



6 · Comparison-Counting Sort



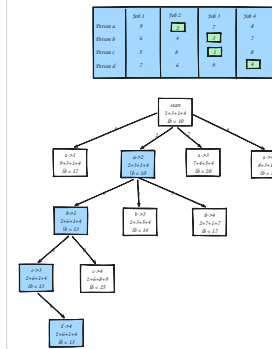
7 · Coin-Row DP



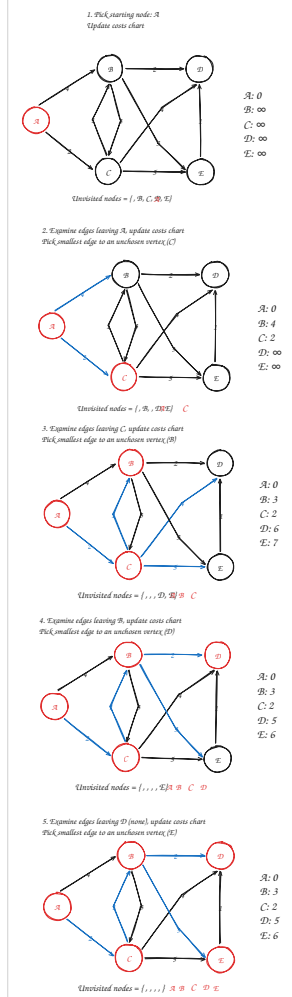
8 · Money-Change DP



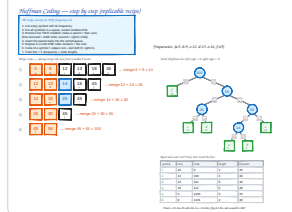
9 · Assignment B&B



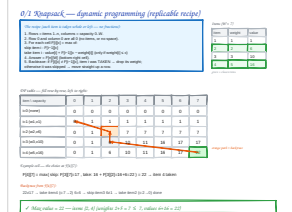
10 · Dijkstra



11 · Huffman Coding



12 · 0/1 Knapsack DP



Algorithm Design & Analysis — Final Cheat Sheet

Design-Technique Map

Technique	Relationship	Signature algos
Decrease & Conquer (U5)	Solve ONE smaller instance, extend to full	Topological sort, insertion sort, binary search
Divide & Conquer (U6)	Split into SEVERAL subproblems, solve each & combine	Quicksort/Hoare, mergesort
Transform & Conquer (U7)	Transform to simpler instance / other representation / other problem, then solve	Heapify, presorting
Space–Time trade-off (U8)	Precompute/store extra data for fast queries	Counting sort, Horspool, hashing
Dynamic Programming (U9)	Overlapping subproblems solved once, stored in table (bottom-up)	Coin-row, coin-change, 0/1 knapsack, Floyd, Warshall
Greedy (U10)	Sequence of locally-best, unalterable choices	Prim, Kruskal, Dijkstra, Huffman
Backtracking / B&B	Build state-space tree with bounds	Assignment problem

DP vs D&C: both split — but D&C subproblems are **independent**; DP exists **because** subproblems overlap (naive recursion is exponential). **Greedy vs DP:** greedy optimizes the next step & commits (fast, only correct with optimal substructure/greedy-choice); DP considers all choices via a table and optimizes the whole path.

Complexity Reference

Algorithm	Best	Avg	Worst	Space
Insertion sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$
Binary search	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(1)$
Topological sort (DFS)	$\Theta(V+E)$	$\Theta(V+E)$	$\Theta(V+E)$	$\Theta(V)$
Mergesort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$
Quicksort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(\log n)$
Binary-tree traversal	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Heapsort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(1)$
Heapify (bottom-up)	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
AVL tree op	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(n)$
Horner's rule	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
Gaussian elimination	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$
Counting sort (cmp / distr)	$\Theta(n^2)/\Theta(n+r)$	$\Theta(n^2)/\Theta(n+r)$	$\Theta(n^2)/\Theta(n+r)$	$\Theta(n)$
Horspool / Boyer–Moore	$\Theta(n/m)$	$\Theta(n)$	$\Theta(nm)$	$\Theta(\alpha)$
Hashing	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Warshall (trans. closure)	$\Theta(n^3)$	$\Theta(n^3)$	$\Theta(n^3)$	$\Theta(n^2)$
Floyd (all-pairs SP)	$\Theta(n^3)$	$\Theta(n^3)$	$\Theta(n^3)$	$\Theta(n^2)$
0/1 knapsack (DP)	$\Theta(nW)$	$\Theta(nW)$	$\Theta(nW)$	$\Theta(nW)$
Prim (min-heap+adj)	$\Theta(E \log V)$	$\Theta(E \log V)$	$\Theta(E \log E)$	$\Theta(V)$
Kruskal	$\Theta(E \log E)$	$\Theta(E \log E)$	$\Theta(E \log E)$	$\Theta(V)$
Dijkstra (min-heap)	$\Theta(E \log V)$	$\Theta(E \log V)$	$\Theta(E \log V)$	$\Theta(V)$
Huffman coding	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$

$\Theta(nW)$ is **pseudo-polynomial** — W is a value, not the input size.

1 · Topological Sort — Source Removal

On a **DAG**, produce a linear order where every edge $u \rightarrow v$ has u before v (like getting dressed: socks before shoes).

Steps: ① find a *source* (no incoming edges) ② output it & delete it + its out-edges ③ repeat until empty.

2 · Array Partition — Hoare

Engine inside quicksort (D&C). Pivot $p=A[l]$; $i=l$, $j=r+1$. Scan **i right** to first element $\geq p$; scan **j left** to first $\leq p$. If $i < j$ swap $A[i], A[j]$ & continue. When $i \geq j$, swap pivot $A[l] \leftrightarrow A[j]$. j is the split point: sp left, $\geq p$ right.

3 · Heap Construction — Bottom-up Heapify

Max-heap = complete binary tree (array) with every parent $\geq$ children; built in $\Theta(n)$ (transform & conquer). For i from last parent $((n/2)-1)$ down to 0: **sift down** — swap with larger child while a child is bigger; stop when $\geq$ both children or a leaf.

4 · MST — Kruskal

MST of weighted connected graph is acyclic with exactly $|V|-1$ edges (cheapest road network). **Steps:** ① sort edges by increasing weight ② add each edge if it does **not** form a cycle ③ stop at $|V|-1$ edges. Closing a cycle means its heaviest edge is never needed.

5 · MST — Prim

Greedy **by vertex**: grow one connected tree from any start, each step adding the cheapest edge from the tree V_T to a new vertex; repeat until all in $(|V|-1)$ edges). **Prim vs Kruskal:** Prim keeps one growing tree; Kruskal takes the globally cheapest edge and may merge several fragments.

6 · Comparison-Counting Sort

Space–time trade-off: count how many elements each one beats. For every pair $i < j$: if $A[i] < A[j]$ then $\text{Count}[j]++$ else $\text{Count}[i]++$. Then $\text{Count}[i] = \#$ elements smaller than $A[i]$ = its final index; place $A[i]$ at $\text{output}[\text{Count}[i]]$. $\Theta(n^2)$.

7 · Coin-Row (DP)

Row of coins $c_1 \dots c_n$; max total, no two adjacent. $F(i) = \max$ value from first i coins. Take $i \rightarrow c_i + F(i-2)$; skip $i \rightarrow F(i-1)$. **$F(i) = \max(F(i-1), c_i + F(i-2))$** , $F(0)=0$, $F(1)=c_1$. Backtrack from the end, jump 2 when a coin is taken.

8 · Money Change (min coins DP)

Fewest coins to make amount m , indexed by **amount**. Ask "what was the last coin c_i ?" $\rightarrow$ before it we had $m-c_i$. **$F(m) = \min \text{ over } c_i \text{ of } F(m-c_i) + 1$** , $F(0)=0$. The $+1$ is the coin placed now; $F(m-c_i)$ solves what's left.

9 · Assignment — Branch & Bound

Assign n people to n jobs, minimize cost (brute force $n!$). ① **Lower bound** = sum of each person's cheapest job ② **branch**: pick next person, child per free job ③ compute each child's bound, explore smallest first, **prune** nodes with bound $\geq$ best complete solution ④ update best when a leaf beats the incumbent.

10 · Dijkstra — single-source SP

Greedy ripples from source (non-negative weights). ① $\text{dist}(\text{src})=0$, others $=\infty$, keep prev pointers ② pop unvisited min-distance vertex (min-PQ), mark visited/final ③ **relax** edges: if $\text{dist}(u) + w(u,v) < \text{dist}(v)$ update $\text{dist}(v)$, $\text{prev}(v)=u$ ④ repeat until all visited.

11 · Huffman Coding

Optimal prefix tree — short codes for frequent chars (greedy). ① leaf per char with frequency in a min-heap ② extract two smallest, make internal node $\text{freq} = \text{sum}$, children become 0/1, reinsert ③ repeat to one root ④ codes = path: left 0, right 1. Rarest chars end deepest.

12 · 0/1 Knapsack (DP)

Items (weight, value), capacity W ; max value, each item once. $F[i][c] = \text{best with first } i \text{ items, capacity } c$. Exclude: $F[i-1][c]$. Include (if $w_i \leq c$): $\text{value}_i + F[i-1][c-w_i]$. **$F[i][c] = \max(F[i-1][c], \text{value}_i + F[i-1][c-w_i])$** . $\Theta(nW)$.

13 · Floyd–Warshall

All-pairs shortest paths, negative edges allowed (no negative cycles). Let paths use intermediate vertices $1 \dots k$: $D_k[i][j] = \min(D_{k-1}[i][j], D_{k-1}[i][k] + D_{k-1}[k][j])$. **Floyd** = shortest distances; **Warshall** = transitive closure (same recurrence with OR/AND). $\Theta(n^3)$.